Design and implement a CNN for Image Classification a) Select a suitable image classification dataset (medical imaging, agricultural, etc.). b) Optimized with different hyper-parameters including learning rate, filter size, no. of layers, optimizers, dropouts, etc.

Code :

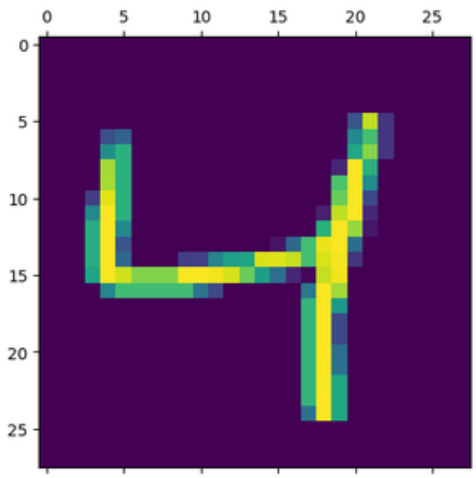
```
import pandas as pd
from tensorflow import keras
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np
from tensorflow.keras import layers, models

(x_train,y_train),(x_test,y_test) = keras.datasets.mnist.load_data()

x_train[1] # it is in 2D array where 0 --> black 252--> white
```

```
array([[  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,   0,  51, 159, 253, 159,  50,   0,   0,   0,   0,   0,   0,
          0,   0],
       [  0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
          0,  48, 238, 252, 252, 252, 237,   0,   0,   0,   0,   0,   0,
          0,   0]
```

```
plt.matshow(x_train[2])
```



Scaling in Machine Learning refers to the process of adjusting the range of features in your data so that they are on a similar scale.

```
x_train = x_train / 255
x_test = x_test / 255
x_test
```

```
array([[[0., 0., 0., ..., 0., 0., 0.],
        [0., 0., 0., ..., 0., 0., 0.],
        [0., 0., 0., ..., 0., 0., 0.],
        ...,
        [0., 0., 0., ..., 0., 0., 0.],
        [0., 0., 0., ..., 0., 0., 0.],
        [0., 0., 0., ..., 0., 0., 0.]],

       [[0., 0., 0., ..., 0., 0., 0.],
        [0., 0., 0., ..., 0., 0., 0.],
        [0., 0., 0., ..., 0., 0., 0.],
        ...,
        [0., 0., 0., ..., 0., 0., 0.],
        [0., 0., 0., ..., 0., 0., 0.],
        [0., 0., 0., ..., 0., 0., 0.]],

       [[0., 0., 0., ..., 0., 0., 0.],
        [0., 0., 0., ..., 0., 0., 0.]
```

Create Validation Split

```
from sklearn.model_selection import train_test_split

x_train, x_val, y_train, y_val = train_test_split(
    x_train, y_train, test_size=0.2
)
```

Build CNN Model (Tunable Params)

```
# change these values to try different hyperparameters
filters = 64
kernel_size = 3
dense_units = 128
dropout_rate = 0.3
learning_rate = 0.001

model = models.Sequential([
    layers.Conv2D(filters, (kernel_size, kernel_size), activation='relu', input_shape=(28,28,1)),
    layers.MaxPooling2D(),

    layers.Conv2D(filters, (kernel_size, kernel_size), activation='relu'),
    layers.MaxPooling2D(),

    layers.Flatten(),
    layers.Dense(dense_units, activation='relu'),
    layers.Dropout(dropout_rate),

    layers.Dense(10, activation='softmax')
])
```

```
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=learning_rate),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

history = model.fit(
    x_train, y_train,
    epochs=10,
    validation_data=(x_val, y_val)
)
```

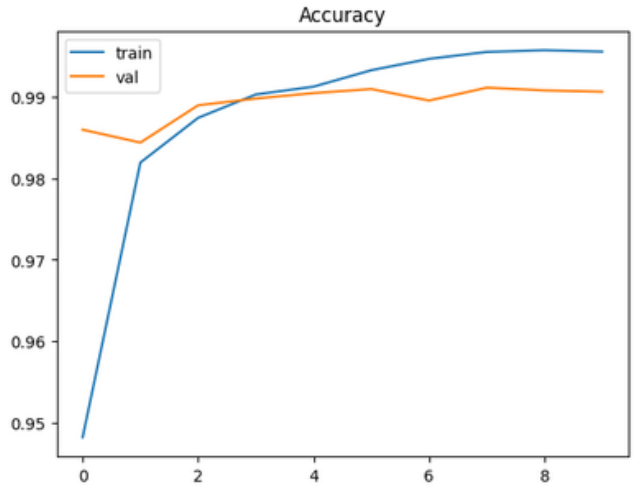
```
Epoch 1/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 18s 11ms/step - accuracy: 0.8850 - loss: 0.3690 - val_accuracy: 0.9860 - val_loss: 0.0
490
Epoch 2/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 15s 10ms/step - accuracy: 0.9816 - loss: 0.0599 - val_accuracy: 0.9844 - val_loss: 0.0
497
Epoch 3/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 15s 10ms/step - accuracy: 0.9871 - loss: 0.0424 - val_accuracy: 0.9890 - val_loss: 0.0
374
Epoch 4/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 15s 10ms/step - accuracy: 0.9913 - loss: 0.0291 - val_accuracy: 0.9898 - val_loss: 0.0
355
Epoch 5/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 15s 10ms/step - accuracy: 0.9925 - loss: 0.0234 - val_accuracy: 0.9905 - val_loss: 0.0
333
Epoch 6/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 15s 10ms/step - accuracy: 0.9941 - loss: 0.0175 - val_accuracy: 0.9910 - val_loss: 0.0
340
Epoch 7/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 15s 10ms/step - accuracy: 0.9957 - loss: 0.0146 - val_accuracy: 0.9896 - val_loss: 0.0
364
Epoch 8/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 14s 10ms/step - accuracy: 0.9962 - loss: 0.0114 - val_accuracy: 0.9912 - val_loss: 0.0
349
Epoch 9/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 15s 10ms/step - accuracy: 0.9962 - loss: 0.0125 - val_accuracy: 0.9908 - val_loss: 0.0
440
Epoch 10/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 14s 9ms/step - accuracy: 0.9957 - loss: 0.0124 - val_accuracy: 0.9907 - val_loss: 0.04
30
```

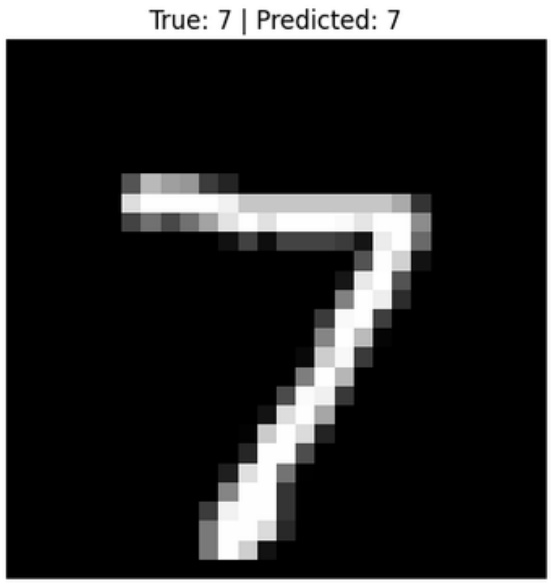
```
test_loss, test_acc = model.evaluate(x_test, y_test)

print("Test Accuracy:", test_acc)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9900 - loss: 0.0388
Test Accuracy: 0.9914000034332275
```

```
plt.plot(history.history['accuracy'], label='train')
plt.plot(history.history['val_accuracy'], label='val')
plt.legend()
plt.title("Accuracy")
plt.show()
```




```
# pick one sample from test set
index = 0  # you can change this

# get image and true label
img = x_test[index]
true_label = y_test[index]

# reshape for prediction (add batch dimension)
img_input = np.expand_dims(img, axis=0)

# prediction
pred = model.predict(img_input)
pred_label = np.argmax(pred)

# plot image
plt.imshow(img.squeeze(), cmap='gray')  # squeeze removes extra channel
plt.title(f"True: {true_label} | Predicted: {pred_label}")
plt.axis('off')
plt.show()
```